

Assignment 7.1

2303A51250

L.Indu

Batch – 18

Task Description #1:

(Syntax Errors – Missing Parentheses in Print Statement)

Task: Provide a Python snippet with a missing parenthesis in a print statement (e.g., `print "Hello"`). Use AI to detect and fix the syntax error.

Bug: Missing parentheses in print statement

```
def greet():
```

```
print "Hello, AI Debugging Lab!"
```

```
greet()
```

Requirements:

- Run the given code to observe the error.
- Apply AI suggestions to correct the syntax.
- Use at least 3 assert test cases to confirm the corrected code works.

Expected Output #1:

- Corrected code with proper syntax and AI explanation.

Prompt :

- 1.The AI correctly identified the syntax error and fixed it.
- 2.The corrected code is clear, simple, and works without errors.

```

task7.py > ...
1  #The following Python code has a syntax error because of a missing parenthesis in a print statement
2  # Identify the error and correct the code. Also add test cases to verify the corrected function."
3  # Bug: Missing parentheses in print statement
4  #def greet():
5  |   # print "Hello, AI Debugging Lab!"
6
7  #greet()
8  # Corrected code with parentheses in the print statement
9  def greet():
10 |   return "Hello, AI Debugging Lab!"
11
12 print(greet())
13 # Test case 1
14 assert greet() == "Hello, AI Debugging Lab!"
15
16 # Test case 2
17 assert isinstance(greet(), str)
18
19 # Test case 3
20 assert "AI" in greet()

```

Output

```

PS C:\Users\ANIL\Desktop\AI Assisant> & C:/Users/ANIL/AppData/Local/Python/pythoncore-3.14-64/python.exe "c:/Users/ANIL/Desktop/AI Assisan
t/task7.py"
Hello, AI Debugging Lab!
PS C:\Users\ANIL\Desktop\AI Assisant>

```

Analysis

- I gave a simple and clear prompt to the AI tool.
- The AI generated a correct class with constructor and display method.
- The code was easy to understand and executed without errors.

Task Description #2:

(Incorrect condition in an If Statement)

Task: Supply a function where an if-condition mistakenly uses = instead of ==. Let AI identify and fix the issue.

Bug: Using assignment (=) instead of comparison (==)

```
def check_number(n):
```

```
if n = 10:
```

```
return "Ten"
```

```
else:
```

```
return "Not Ten"
```

Requirements:

- Ask AI to explain why this causes a bug.
- Correct the code and verify with 3 assert test cases.

Expected Output #2:

- Corrected code using == with explanation and successful test execution.

Prompt :

Bug: Using assignment (=) instead of comparison (==)

```
#def check_number(n):
```

```
    # if n = 10:
```

```
        # return "Ten"
```

```
    #else:
```

```
        #return "Not Ten"
```

#The following Python function has a bug because it uses the assignment operator instead of the comparison operator in an if condition. Identify the error, explain why it occurs, and correct the code. Also verify the corrected code using assert test cases.

Corrected code with comparison operator

Code:

```

task72.py > ...
1 # Bug: Using assignment (=) instead of comparison (==)
2 #def check_number(n):
3     # if n = 10:
4         # return "Ten"
5     #else:
6         #return "Not Ten"
7 #The following Python function has a bug because it uses the assignment operator instead of the comparison operator in an if condition. Identify the error, explain
8 # Corrected code with comparison operator
9 def check_number(n):
10     if n == 10:
11         return "Ten"
12     else:
13         return "Not Ten"
14 print(check_number(10))
15 print(check_number(5))
16 print(check_number(0))
17 # Test case 1
18 assert check_number(10) == "Ten"
19 # Test case 2
20 assert check_number(5) == "Not Ten"
21 # Test case 3
22 assert check_number(0) == "Not Ten"
23

```

Output:

```

PS C:\Users\ANIL\Desktop\AI Assisant> & C:/Users/ANIL/AppData/Local/Python/pythoncore-3.14-64/python.exe "c:/Users/ANIL/Desktop/AI Assisant/task72.py"
Ten
Not Ten
Not Ten
PS C:\Users\ANIL\Desktop\AI Assisant>

```

Analysis:

1. The error occurred because = is an assignment operator, not a comparison operator.
2. Python does not allow assignment inside an if condition.
3. Replacing = with == fixed the logic error.

Task Description #3:

(Runtime Error – File Not Found)

Task: Provide code that attempts to open a non-existent file and crashes. Use AI to apply safe error handling.

Bug: Program crashes if file is missing

```
def read_file(filename):
```

```
    with open(filename, 'r') as f:
```

```
        return f.read()
```

```
print(read_file("nonexistent.txt"))
```

Requirements:

- Implement a try-except block suggested by AI.
- Add a user-friendly error message.
- Test with at least 3 scenarios: file exists, file missing, invalid path.

Expected Output #3:

- Safe file handling with exception management.

Prompt :

Python code crashes when the file does not exist.

#Identify the runtime error, explain why it occurs, and rewrite the code using proper try-except blocks to handle missing files safely.

#Add a user-friendly error message and test the program with multiple scenarios.

#def read_file(filename):

 #with open(filename, 'r') as f:

 #return f.read()

#print(read_file("nonexistent.txt"))

Corrected code with try-except block to handle missing files

```
> task73.py > read_file
1  # Python code crashes when the file does not exist.
2  #Identify the runtime error, explain why it occurs, and rewrite the code using proper try-except blocks to handle missing files safely.
3  #Add a user-friendly error message and test the program with multiple scenarios.
4  #def read_file(filename):
5      #with open(filename, 'r') as f:
6          #return f.read()
7  #print(read_file("nonexistent.txt"))
8  # Corrected code with try-except block to handle missing files
9  def read_file(filename):
10     try:
11         with open(filename, 'r') as f:
12             return f.read()
13     except FileNotFoundError:
14         return "Error: File not found."
15     except Exception:
16         return "Error: Invalid file path or unable to read file."
17  # Test case 1: File does not exist
18  print(read_file("nonexistent.txt")) # Expected output: "Error: File not found"
19  # Test case 2: Valid file path (assuming 'example.txt' exists with some content)
20  print(read_file("example.txt")) # Expected output: Content of example.txt
21  # Test case 3: Invalid file path
22  print(read_file("/invalid/path/to/file.txt")) # Expected output: "Error: Invalid file path or unable to read file."
23
```

Output:

```
PS C:\Users\ANIL\Desktop\AI Assisant> & C:/Users/ANIL/AppData/Local/Python/pythoncore-3.14-64/python.exe "c:/Users/ANIL/Desktop/AI Assisant/task73.py"
Error: File not found.
Error: File not found.
Error: File not found.
PS C:\Users\ANIL\Desktop\AI Assisant> 
```

Analysis:

1. The program crashed because the file was not available.
2. The AI suggested using a try-except block to handle the error.
3. FileNotFoundError was handled with a user-friendly message.

Task Description #4:

(Calling a Non-Existent Method)

Task: Give a class where a non-existent method is called (e.g., `obj.undefined_method()`). Use AI to debug and fix.

Bug: Calling an undefined method

```
class Car:
```

```
    def start(self):
```

```
        return "Car started"
```

```
my_car = Car()
```

```
print(my_car.drive()) # drive() is not defined
```

Requirements:

- Students must analyze whether to define the missing method or correct the method call.
- Use 3 assert tests to confirm the corrected class works.

Expected Output #4:

- Corrected class with clear AI explanation.

Prompt :

```
#class Car:
```

```
    # def start(self):
```

```
        # return "Car started"
```

```
#my_car = Car()
```

```
#print(my_car.drive()) # drive() is not defined
```

#The following Python class throws an error because a non-existent method is called.

#Identify the error, explain why it occurs, and correct the code by either defining the missing method or fixing the method call.

#Verify the corrected class using assert test cases

Code :

```
task74.py > ...
1  #class Car:
2      # def start(self):
3          # return "Car started"
4
5  #my_car = Car()
6  #print(my_car.drive()) # drive() is not defined
7  #The following Python class throws an error because a non-existent method is called.
8  #Identify the error, explain why it occurs, and correct the code by either defining the missing method or fixing the method call.
9  #Verify the corrected class using assert test cases
10 class Car:
11     def start(self):
12         return "Car started"
13
14     def drive(self):
15         return "Car is driving"
16 my_car = Car()
17
18 # Test case 1
19 assert my_car.start() == "Car started"
20
21 # Test case 2
22 assert my_car.drive() == "Car is driving"
23
24 # Test case 3
25 assert isinstance(my_car, Car)
26
```

Output

```
PS C:\Users\ANIL\Desktop\AI Assisant> & C:/Users/ANIL/AppData/Local/Python/pythoncore-3.14-64/python.exe "c:/Users/ANIL/Desktop/AI Assisant/task74.py"  
PS C:\Users\ANIL\Desktop\AI Assisant> █
```

Analysis:

1. The error happened because a method that does not exist was called.
2. Python cannot find the drive() method in the class.
3. The AI suggested adding the missing method.

Task Description #5:

TypeError – Mixing Strings and Integers in Addition)

Task: Provide code that adds an integer and string ("5" + 2) causing a TypeError. Use AI to resolve the bug.

Bug: TypeError due to mixing string and integer

```
def add_five(value):
```

```
    return value + 5
```

```
print(add_five("10"))
```

Requirements:

- Ask AI for two solutions: type casting and string concatenation.
- Validate with 3 assert test cases.

Expected Output #5:

- Corrected code that runs successfully for multiple inputs.

Note: Report should be submitted a word document for all tasks in a single document with prompts, comments & code explanation, and

output and if required, screenshots

Prompt :

```
#def add_five(value):
```

```
    # return value + 5
```

```
#print(add_five("10"))
```

#The following Python function causes a TypeError because it tries to add a string and an integer.

#Identify the error, explain why it occurs, and fix the code using:

#Type casting

#String concatenation

#Verify the corrected code using assert test cases.

Corrected code using type casting

Code :

```
task75.py > ...
1  #def add_five(value):
2  |     # return value + 5
3
4  #print(add_five("10"))
5  #The following Python function causes a TypeError because it tries to add a string and an integer.
6  #Identify the error, explain why it occurs, and fix the code using:
7
8  #Type casting
9
10 #String concatenation
11 #Verify the corrected code using assert test cases.
12 # Corrected code using type casting
13 def add_five(value):
14 |     return int(value) + 5
15 assert add_five("10") == 15
16 assert add_five(5) == 10
17 assert add_five("0") == 5
18 # Corrected code using string concatenation
19 def add_five(value):
20 |     return str(value) + "5"
21 assert add_five("10") == "105"
22 assert add_five(5) == "55"
23 assert add_five("0") == "05"
```

Output :

```
PS C:\Users\ANIL\Desktop\AI_Assisant> & C:/Users/ANIL/AppData/Local/Python/pythoncore-3.14-64/python.exe "c:/Users/ANIL/Desktop/AI_Assisant/task74.py"
PS C:\Users\ANIL\Desktop\AI_Assisant> & C:/Users/ANIL/AppData/Local/Python/pythoncore-3.14-64/python.exe "c:/Users/ANIL/Desktop/AI_Assisant/task75.py"
PS C:\Users\ANIL\Desktop\AI_Assisant> 
```

Analysis:

1. The error happened due to mixing string and integer values.
2. Python does not allow adding different data types directly.
3. The AI suggested type casting and string concatenation as fixes.